Our Modifications

KMA DKMA GNN-KMA

red red red

Algorithm Design, Synthetic Dataset Generation & GNN-Guided Hybrid Solvers

KMA · DKMA · GNN-KMA v1 / v2 / v3

CSE 4-2 Algorithm Project · April 10, 2026

Outline

Feedback Vertex Set — Problem Statement

Definition

Given a graph $G = (V, E)$, find the **smallest** set $S \subseteq V$ such that $G \setminus S$ is *acyclic*.

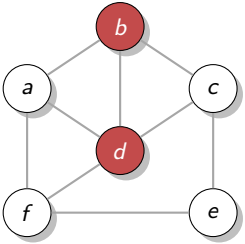
Directed FVS (DFVS): $G \setminus S$ is a DAG.

NP-Hardness

FVS is NP-hard in general; it is one of Karp's original 21 NP-complete problems [**?**]. DFVS is also NP-hard [**?**].

Parameterized Complexity

FVS is fixed-parameter tractable in k (solution size): $O(4^k \cdot k! \cdot n^{O(1)})$ [**?**]. Modern kernelization yields a $O(k^2)$-vertex kernel [**?**].

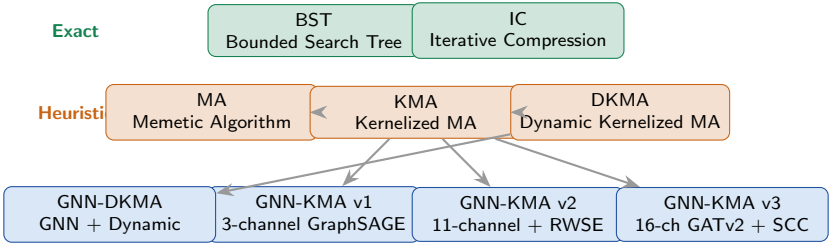


Red nodes = FVS $\{b, d\}$

Removal leaves a forest

Applications:
Deadlock recovery, VLSI design,
Bayesian network inference,
circuit testing, bioinformatics

Project Scope & Algorithm Hierarchy



Exact Solvers: BST & IC

Bounded Search Tree (BST)

- Pick a cycle C; branch on every vertex of C
- At each branch, include that vertex in the FVS
- Prune when solution size $> k$
- Complexity: $O(4^k \cdot n)$ [**?**]
- Combined with **reduction rules**: self-loops, degree-1 bypass, degree-2 bypass
- Applicable only for small k (exact track)

Iterative Compression (IC)

- Introduced by Reed et al. [**?**]
- Maintains a solution S of size $k+1$; iteratively **compresses** to size k
- Uses important separator enumeration
- Yields $O(5^k \cdot k \cdot n^2)$ time [**?**]
- Provides **exact labels** for GNN training on the exact track
- Correctness enforced via acyclicity check on final solution

Usage in this project: IC is used as the **label oracle** for exact-track GNN training data. BST is a secondary exact baseline. Both rely on the parameterized FVS kernel theory of Thomassé [**?**], which reduces any instance to a $O(k^2)$-vertex kernel.

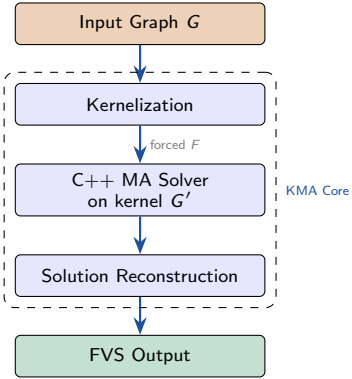
KMA — Motivation and High-Level Design

Why not pure Memetic Algorithm?

- A plain MA [**?**] operates on the full graph
- Reduction rules can *safely* eliminate many vertices before any search [**?**]
- This shrinks the search space, letting MA focus on hard subgraphs

KMA Pipeline

1. **Kernelization** — apply reduction rules, record forced vertices
2. **MA on kernel** — C++ evolutionary search on reduced graph
3. **Reconstruction** — map kernel solution to original indices, union with forced vertices



KMA — Kernelization Rules

Undirected Kernelization Rules

1. **Self-loop:** vertex with self-loop $\Rightarrow$ force into FVS
2. **Degree-0/1:** acyclic leaf $\Rightarrow$ safely delete
3. **Degree-2 bypass:** if $\deg(v) = 2$ with neighbors a, b and $\{a, b\} \notin E$: delete v, add edge (a, b)

Rules applied **iteratively** until quiescence.

Theoretical basis: $O(k^2)$ kernel [**?**].

Directed Kernelization Rules

1. **Self-loop:** forced into DFVS
2. **Source/Sink:** in-deg $= 0$ or out-deg $= 0 \Rightarrow$ delete (not on any directed cycle)
3. **Degree-1 bypass:** in-deg or out-deg $= 1 \Rightarrow$ shortcut edges, delete vertex
4. **SCC reduction:** retain only vertices in nontrivial SCCs (size > 1 or self-loop)

Directed kernel theory: [**?**].

Modification over baseline MA: The addition of kernelization (especially the SCC-based directed reduction) is a key novelty. SCC reduction alone can eliminate **60–90%** of vertices on sparse real-world directed graphs before any search begins.

KMA — Memetic Algorithm Core

C++ MA Solver (evolutionary search on kernel):

- Initialise population of size P randomly
- Each individual: binary vector over kernel vertices
- Each generation:
  1. **Crossover** (two parents → offspring)
  2. **Local mutation** (flip, swap)
  3. **Repair**: restore FVS validity via greedy cycle cover
  4. **Selection**: tournament
- Early-stop if no improvement for T generations
- Hard wall-clock timeout enforced inside C++

Solver dispatch priority (best-first): `KMA` → `KME`

Parameters

Param	Flag	Default
Population	`--pop`	20
Max generations	`--gens`	100
Timeout	`--timeout`	600s
Early-stop	`--earlystop`	30

Complexity Note

Kernelization: $O(n + m)$ per pass
MA on kernel G': heuristic,
$O(P \cdot \text{gen} \cdot |V(G')|)$
Full pipeline: practical for
$|V| \leq 10^4$ within timeout

DKMA — Core Idea

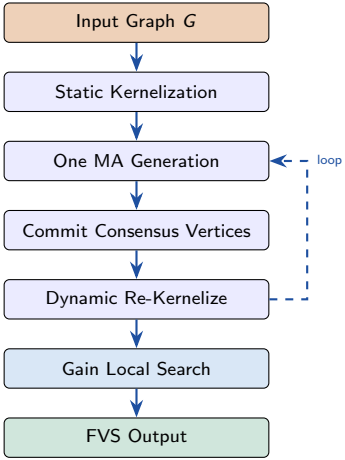
Limitation of static kernelization (KMA):

- Kernelization is applied *once* before MA
- Partial solutions during MA may *unlock* new reductions — these are never exploited

DKMA — Dynamic interleaving [**?**, **?**]:

1. Evolve population for one MA generation
2. **Commit** vertices appearing in $\geq \tau$ fraction of individuals
3. Force committed vertices into the solution
4. **Re-kernelize** residual graph
5. Continue MA on the smaller kernel

This **opening-up effect** progressively shrinks the kernel as partial agreement is reached.



DKMA — Key Innovations & Helpers

SHORTONE Directed Contractions

Inspired by the Mount-Doom PACE 2022 solver [?], a directed contraction pass (`_apply_shortone_rule`) is applied before the MA loop on directed graphs.

Contracts degree-1 in/out vertices with special cycle-preserving rewiring, enabling additional SCC reductions.

Topological Diversification

Every 5 generations, `_diversify_with_topological_ordering` is called:

- Directed: randomised Kahn's algorithm tie-breaking

Gain Local Search (post-loop)

`_gain_local_search` applies **1-1** and **2-1** improvement swaps after the MA loop:

- **1-1 swap**: remove one FVS vertex, add another if acyclicity is preserved and size decreases
- **2-1 swap**: replace two FVS vertices with one

Commit Threshold & Cap

`_commit_vertices_from_population`:

- Commit if vertex appears in $\geq \tau$ of population
- Cap at top 30% by frequency if

DKMA — Correctness & Comparison with KMA

Correctness Safeguards

- Deduplicated final solution indices
- All forced vertices (static + dynamic) included
- Final **acyclicity verification** on original graph: directed ⇒ iterative DFS back-edge detection; undirected ⇒ union-find cycle check
- **Fallback to KMA** if validation fails

KMA vs DKMA — Key Differences

Feature	KMA	DKMA
Kernelization	Once	Per loop
Commitment	None	Consensus
SHORTONE pass	No	Yes (directed)
Topological div.	No	Every 5 gen.
Local search	No	Gain 1-1/2-1
Acyclicity check	End	End + fallback

References: DKMA design is inspired by the PACE 2022 FVS track winner strategies [?] (Grossmann, Heuer, Schulz, Strash, IPEC 2022) and the ESA 2022 DFVS analysis [?] (Figiel, Froese, Nichterlein, Niedermeier). The Mount-Doom SHORTONE rule is from Langedal et al. [?].

GNN-KMA — Hybrid Coupling Strategy

Motivation:

- GNN learns vertex-level patterns from solved instances
- Predicts probability $p(v)$ of belonging to FVS
- High-confidence predictions **hard-fix** vertices before KMA, reducing the search problem

Precision-first soft-hint coupling:

1. Kernelize graph $G \to G'$
2. Run GNN on G', get $p(v)$ for each v
3. Hard-fix only vertices with $p(v) \geq \theta$
4. Cap hard-fix at **8%** of kernel size
5. Run KMA on residual kernel
6. Reconstruct: forced $\cup$ GNN-fixed $\cup$ KMA

Why Precision-First?

False positives in hard-fixing are **catastrophic**: a wrongly fixed vertex inflates the FVS size permanently within that branch. Capping at 8% and requiring $\theta \geq 0.65$ ensures the GNN only commits when highly confident.

If no vertex exceeds θ, the system **falls back to pure KMA** automatically.

GNN-DKMA variant: Same coupling, but the residual kernel is handed to DKMA instead of KMA, enabling dynamic re-kernelization atop GNN guidance.

GNN-KMA v1 — Baseline Architecture

Feature Engineering (3 channels)

Undirected:

- Normalised degree $d(v)/d_{\max}$
- Clustering coefficient $c(v)$ [**?**]
- Normalised log-degree

Directed:

- Normalised in-degree
- Normalised out-degree
- Normalised $\min(\text{in}, \text{out})$-degree

Design Choice

Clustering coefficient [**?**] captures local cycle density — vertices with high $c(v)$ are strong FVS candidates. This is a well-established heuristic from the small-world network literature.

Model Architecture

UndirectedFVSNet:

- 3-layer **GraphSAGE** message passing [**?**]
- BatchNorm + Dropout after each layer
- MLP classification head
- 2-class `log-softmax` output

DirectedFVSNet:

- Custom directed aggregation layer (separate in / out / self terms)
- 3 layers + BN + Dropout + MLP
- 2-class `log-softmax` output

Training Objective

GNN-KMA v2 — Richer Structural Features

Feature Engineering (11 channels)

1. Degree-like channels (in/out/min or undirected)
2. **RWSE** — Random Walk Structural Encoding [?]

   short-range return probabilities at steps $\{2, 3, 4\}$
3. **Motif counts**: triangle count, 4-cycle participation, 4-clique style counts [?]
4. **k-core number** $\kappa(v)$ [?]

Architecture & Training

UndirectedFVSNetV2 / DirectedFVSNetV2:

- Same GraphSAGE-style backbone as v1
- Upgraded input: `in_channels=11`
- Richer representation $\Rightarrow$ stronger structural discrimination

Training: same NLL + cosine schedule; checkpoint by validation F1.

Why These Features?

RWSE [?]: captures short-range cycle return probability — directly relevant to FVS membership.

k-core [?]: vertices deep in dense cores are more likely to be in any FVS.

Motif counts [?]: triangle/4-cycle participation indicates cycle exposure.

GNN-KMA v3 — Attention-Based Precision Model

Feature Engineering (16 channels)

- Extended **RWSE** horizon: steps $\{2, 3, 4, 6, 8, 12, 16\}$ [**?**]
- **SCC-aware channels** for directed cycle structure (SCC size, SCC membership depth)
- Cycle-score interaction features
- Directional ratio features $(\text{in}/(\text{in} + \text{out}))$
- Reduced reliance on expensive motif channels

SCC Channels (Directed)

SCC decomposition [?] provides structural depth: vertices in large SCCs are more critical for DFVS. SCC-size and SCC-depth channels directly encode this.

DirectedFVSNetV3 Architecture

- Input projection layer
- **5 residual GATv2 blocks** [?]
- Separate **forward** and **reverse** message paths
- Per-layer fusion projections
- **Global readout context** concatenated to node embedding
- Single-logit output per node (sigmoid)

v3 Training Innovations

- **AsymmetricFVSLoss**: penalises false positives more strongly (precision-oriented)

GNN-KMA — Feature & Architecture Evolution Summary

Aspect	**v1**	**v2**	**v3**
Feature channels	3	11	16
Degree features	✓	✓	✓
Clustering coeff. [**?**]	✓	–	–
RWSE [**?**]	–	steps $\{2, 3, 4\}$	steps $\{2..16\}$
Motif counts [**?**]	–	✓	reduced
k-core [**?**]	–	✓	–
SCC channels [**?**]	–	–	✓
Backbone	GraphSAGE [**?**]	GraphSAGE	GATv2 [**?**]
Depth	3 layers	3 layers	5 residual blocks
Bidirectional	No	No	Yes (fwd+rev)
Global readout	No	No	Yes
Loss	NLL	NLL	AsymmetricFVSLoss
Checkpoint metric	Val F1	Val F1	topk-prec@8%
Epochs	100	100	200

Results

Synthetic Dataset PACE 2022 Evaluation

Notation & Terms

- **MA**: Memetic Algorithm — a genetic algorithm with local search
- **KMA**: Kernelized Memetic Algorithm — MA applied on a reduced kernel
- **DKMA**: Dynamic Kernelized Memetic Algorithm — KMA with iterative re-kernelization
- **GNN-KMA**: GNN-augmented KMA — GNN guides/fixes high-confidence vertices for KMA

Dataset — Design Goals & Generation Pipeline

Five design goals:

1. **Controlled distribution** — fixed category proportions
2. **Track-aware difficulty** — separate exact & heuristic tracks
3. **Reproducibility** — deterministic seeding throughout
4. **Restartability** — safe reuse of existing files
5. **Benchmark/train alignment** — PT datasets mirror TXT corpus structure

Three-stage orchestration:

1. `download_real_world.py`

   NetworkX, PyG, OGB, SNAP sources

2. `generate_synthetic.py`

   Barabási–Albert, Watts–Strogatz, ER, grids, trees, DAGs

Directory Contract

```
data/synthetic/
{undirected|directed}/
{exact_track|heuristic_track}/
{category}/
*.txt
```

Track Size Intent

Track	Node range
exact_track	$\approx$ 10–35 nodes
heuristic_track	$\approx$ 100–5000 nodes

Default: 100K undirected + 100K directed
exact ratio = 0.5, seed = 1337

Dataset — Category Distribution

Undirected (uniform weights, 20% each)

Category	Weight	Graph Model
real_world	20%	NetworkX/PyG/OGB/SN
scale_free	20%	Barabási–Albert [?]
small_world	20%	Watts–Strogatz [?]
random_er	20%	Erdős–Rényi [?]
grids_trees	20%	Grid + Random tree

Directed (non-uniform weights)

Category	Weight
real_world_ego	30%
scale_free	20%
random_er	20%
directed_grids	15%
dags	15%

Real-world share:
Undirected: 20% from real-world sources
Directed: 30% from ego-network sources

Sources: NetworkX built-ins, PyG datasets, OGB node property datasets, SNAP archives. Cached under `.cache/real_graphs`.

Label Sources

exact_track: IC solver labels (high-fidelity, near-optimal)

heuristic_track: KMA labels (practical, large-instance behavior)

Dataset — Why This Dataset is Strong

Diversity

- **5 undirected families**: covers scale-free hubs, small-world clusters, random structure, regular grids/trees, and real topology
- **5 directed families**: adds ego networks and DAGs
- Both undirected and directed instances
- Two difficulty regimes (exact / heuristic)

Quality Guarantees

- Each label validated via acyclicity check
- Subprocess timeout isolation for all solver calls
- CSV status ledger tracks completed / timeout / invalid outcomes
- Largest connected component extracted
- Compact integer relabelling applied
- No self-loops in edge lists

Scale

100K total undirected + 100K directed

(20K version also available on Kaggle)

Kaggle (100K):
`kaggle.com/.../fvs-synthetic-dataset-100k`

Kaggle (20K):
`kaggle.com/.../fvs-synthetic-dataset-20k`

PACE 2022 Dataset — Real-World Benchmark Testing

What is PACE 2022?

The **Parameterized Algorithms and Computational Experiments** (PACE) Challenge 2022 [**?**] focused on the **Feedback Vertex Set** problem.

It provided:

- 200 public undirected test instances
- Curated from real-world and synthetic sources
- Difficulty ranges from small exact to large heuristic
- Official solution checker for validation
- World-class solver implementations as baselines

Usage in This Project

- PACE 2022 instances stored under `data/pace2022/`
- **Protected from cleanup** — never deleted by `setup_benchmark_inputs.py`
- Benchmarked via `pacerun.sh` and `paceresults/`
- Used to evaluate **heuristic solvers** (KMA, DKMA, GNN-KMA v1/v2/v3) against real competition-grade instances
- PACE winner results in `pace2022_winner.csv` used as comparison baseline

End-to-End System Architecture

Layer 1: C++ Algorithm Engine (`cpp_engine/`)

BST	IC	MA	KMA	DKMA	Directed variants

Layer 2: Python Orchestration (`experiments/`)

Benchmark drivers	Pipeline runner	Hybrid runtime	Suite runner

Layer 3: Data Pipeline (`data/`, `gnn_model/dataset_gen.py`)

Real-world download	Synthetic generation	TXT→PT labelling	PACE 2022 instances

Layer 4: GNN Stack (`gnn_model/`)

Feature eng. v1/v2/v3	Model definitions	Train loop	Runtime inference

Evaluation Framework

Benchmark Output Format

Each benchmark run produces a CSV with:

- File identity & graph metadata
- Algorithm / family / track tags
- `fvs_size`: solution size
- `runtime`: wall-clock seconds
- `valid`: acyclicity-verified boolean
- `status`: completed / timeout / error

Benchmark Tracks

exact_track: compared against IC/BST optimal

heuristic_track: compared against MA baseline

PACE 2022: compared against official winner

Algorithm Comparison Matrix

Algorithm	Exact	Heuristic
BST	✓	–
IC	✓	–
MA	–	✓
KMA	–	✓
DKMA	–	✓
GNN-KMA v1	–	✓
GNN-KMA v2	–	✓
GNN-KMA v3	–	✓
GNN-DKMA	–	✓

Resume semantics: all benchmark pipelines support deterministic restart — interrupted runs continue from last completed file.

Summary & Contributions

Algorithm Contributions

- **KMA**: first-class kernelization (SCC reduction for directed; degree-2 bypass for undirected) integrated with C++ memetic search
- **DKMA**: dynamic re-kernelization loop, SHORTONE contractions [**?**], topological diversification, gain local search (1-1 / 2-1 swaps)
- **GNN-KMA v1/v2/v3**: progressive feature and architecture improvements (GraphSAGE → GATv2 [**?**], 3 → 16 channels, NLL → asymmetric precision loss)
- **GNN-DKMA**: GNN guidance + dynamic kernelization

Dataset Contributions

- 100K undirected + 100K directed labelled FVS instances
- 5 graph families, 2 difficulty tracks, real+synthetic mix
- Solver-verified ground-truth labels (IC for exact, KMA for heuristic)
- Publicly released on Kaggle (100K and 20K versions)
- PACE 2022 integration for real-world benchmarking

GitHub:
`https://github.com/TAR2003/Feedback_Vertex_Set_Problem_Algorithm_Project_4-2`

References I

- R.M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, 1972, pp. 85–103.
- A. Becker and D. Geiger, "Optimization of Pearl's method of conditioning and greedy-like approximation algorithms for the weighted maximum acyclic subgraph problem," *Artificial Intelligence*, vol. 105, pp. 1–2, 1996.
- S. Thomassé, "A $4k^2$ kernel for feedback vertex set," *ACM Trans. Algorithms*, vol. 6, no. 2, 2010.
- B. Reed, K. Smith, and A. Vetta, "Finding odd cycle transversals," *Operations Research Letters*, vol. 32, no. 4, 2004.
- M. Cygan *et al.*, *Parameterized Algorithms*. Springer, 2015.
- J. Chen, Y. Liu, S. Lu, B. O'Sullivan, and I. Razgon, "A fixed-parameter algorithm for the directed feedback vertex set problem," *J. ACM*, vol. 55, no. 5, 2008.
- T. Lust and J. Teghem, "The multiobjective multidimensional knapsack problem: a survey and a new approach," *Int. Trans. Oper. Res.*, 2010. (Memetic algorithm framework reference.)
- G. Grossmann, M. Heuer, P. Schulz, and D. Strash, "Targeted solution methods for the minimum feedback vertex set problem," in *Proc. 17th IPEC*, LIPIcs vol. 249, 2022, doi:10.4230/LIPIcs.IPEC.2022.26.
- A. Figiel, V. Froese, A. Nichterlein, and R. Niedermeier, "Improved parameterized algorithms for the directed feedback vertex set problem," in *Proc. ESA 2022*, LIPIcs, 2022, doi:10.4230/LIPIcs.ESA.2022.53.
- K. Langedal, J. Langguth, F. Manne, and D. Strash, "Efficient minimum weight vertex cover heuristics using graph neural networks" (Mount-Doom FVS solver), Zenodo, 2022, doi:10.5281/zenodo.6630611.

References II

- W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs" (GraphSAGE), in *NeurIPS*, 2017.
- S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" (GATv2), in *ICLR*, 2022.
- V.P. Dwivedi *et al.*, "Benchmarking graph neural networks," *JMLR*, vol. 24, no. 43, 2023. (RWSE/positional encoding reference.)
- N.K. Ahmed, J. Neville, R.A. Rossi, and N. Duffield, "Efficient graphlet counting for large networks," in *ICDM*, 2015.
- V. Batagelj and M. Zaversnik, "An $O(m)$ algorithm for cores decomposition of networks," *arXiv:cs/0310049*, 2003.
- R.E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, 1972.
- A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, 1999.
- D.J. Watts and S.H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, 1998.
- P. Erdős and A. Rényi, "On random graphs I," *Publicationes Mathematicae*, vol. 6, 1959.

Questions & Discussion